Kwaliteitseisen Programmeerproject

Gereviseerd door Youri Coppens & Louise Van Verre
project2ba@dinf.vub.ac.be

Voorwoord

Dit document is een herwerkte versie van het document 'Kwaliteitseisen Programmeerprojecten Softwareproject en Systeemprogrammatuur 2008-2009'.

Wij danken de originele auteurs voor hun deskundige verwoordingen en adviezen:
Theo D'Hondt — Andy Kellens — Stijn Mostinckx — William Van Woensel — Peter Schelkens — Tim Bruylants — Frederik Temmermans

alsook de redacteurs uit voorgaande jaargangen (2001 t.e.m. 2007):
Sven Castelyn — Wolfgang De Meuter — Kris Gybels — Matthias Stevens

Wij blijven trachten jullie expertise over te dragen.

Inhoudsopgave

Inleiding 1
De Kwaliteitseisen 2
Abstracties maken 2
Abstract Data Type vs. Object 5
Abstract Data Type vs. Datastructuur 6
Booleans Zijn Volwaardige Datatypes 7
Codeduplicatie 7
Performantie en Geheugengebruik 9
Naamgeving en Andere Codeerconventies 10
Geen Vastgebakken Waarden 11
Geen Globale Variabelen 12
Gebruik de Mogelijkheden van De Programmeertaal 12
Korte Procedures 12
Codehergebruik van Derden 13
Scheiding van Belangen 13
Debugging & Testing 15
Documentatie 16
Stijlfouten voor Beginners 16
Samenvatting en Slotopmerkingen 17

Inleiding

In dit document worden een aantal kwaliteitseisen gegeven voor het vak Programming Project 2, deel van het curriculum van het tweede jaar Bachelor in de Computerwetenschappen. Dit document beschrijft enkel kwaliteitseisen voor al de code die je in de loop van dit vak dient in te dienen, voor verdere informatie over de projectinhoud dien je je te wenden tot het Canvas leerplatform, waar je onder andere de deadlines en de opgave van het project vindt.

Globaal genomen zijn er drie belangrijke eisen aan elk softwaresysteem: het moet doen wat het zou moeten doen, het moet dit efficiënt doen, en wanneer het in de toekomst nieuwe taken zou moet kunnen vervullen moet het makkelijk zijn de implementatie aan te passen of uit te breiden. Het eerste is natuurlijk een basisvereiste van het systeem: als het systeem niet doet wat er opgelegd werd, is het gewoon niet nuttig. De laatste twee eisen zijn eerder kwalitatieve eisen die het nut van het systeem vergroten: een systeem waarin geheugen en berekeningstijd efficiënt benut worden vereist geen onnodig dure computerapparatuur om het te gebruiken; een systeem dat makkelijk aan te passen is kan langer mee want hoeft niet van nul geherimplementeerd te worden wanneer het nieuwe taken moet kunnen vervullen.

Wat het systeem van je programmeerproject zou moeten doen wordt reeds beschreven in de projectopdracht. We gaan hier dieper in op de twee globale kwaliteitseisen door enkele concrete eisen te geven waar je systeem zeker aan moet voldoen. Er wordt vooral aandacht geschonken aan de aanpasbaarheidseis, gezien er hier heel wat bij komt kijken: om makkelijk aanpasbaar en uitbreidbaar te zijn moet je code leesbaar zijn, duidelijk in onderdelen verdeeld zijn zodat de impact van aanpassingen kan beperkt worden tot dat onderdeel, de werking van de onderdelen moet duidelijk gecommuniceerd worden naar andere programmeurs toe enz. In de laatste sectie van dit document geven we een korte samenvatting, beschrijven we het relatieve belang van de drie globale eisen en geven we nog enkele slotopmerkingen.

De Kwaliteitseisen

In deze sectie geven we een diepgaandere uitleg van wat “kwalitatief hoogstaande” softwaresystemen schrijven zoal inhoudt. We geven ook een lijst van concreet gestelde, vetgedrukte eisen die je zeker dient te volgen. Lees echter niet enkel deze delen van het document, maar ook de bijbehorende uitleg!

Hoewel de meeste van de onderstaande eisen geschreven zijn met Racket/Scheme in gedachten (de programmeertaal die je verplicht bent te gebruiken voor het project), zijn het eigenlijk stuk voor stuk algemene kwaliteitskenmerken waar code in om het even welke hogere programmeertaal, geschreven door om het even welke programmeur aan zou moeten voldoen.

Abstracties maken

Eis: Gebruik in je code nergens `car` en `cdr`, tenzij in de definitie van een ADT.

Elk jaar opnieuw krijgen we code binnen die vol staat met het gebruik van `car` en `cdr` (en al hun varianten: `cadr`, `caadr`, `cdar`, `first`, `second`...). Dit is bijna steeds te wijten aan het onvolledig begrijpen van waarom en hoe men abstracties gebruikt. De enige plaats waar `car` en `cdr` mogen voorkomen in je code is in de *definitie* van een ADT, *niet* in het gebruik ervan.

Beschouw het volgende voorbeeld waarin uitgelegd wordt hoe je op een correct manier abstraheert. Veronderstel dat men een punt wilt voorstellen in een 3-dimensionale

ruimte en verkiest om gebruik te maken van een lijstvoorstelling. Volgende definitie is dan een mogelijke constructor-procedure:

```racket
(define (make-point x y z)
  (cons x (cons y (cons z '()))))
```

Ergens in de code wordt dan bijvoorbeeld de volgende expressie uitgevoerd met een instantie genaamd `point`:

```racket
(if (= (car point) 1)
    (displayln "x-coördinaat is 1")
    (displayln "x-coördinaat is niet 1"))
```

Het probleem hier is dat jij als programmeur weet dat een punt wordt voorgesteld als een lijst van 3 elementen, en dat het eerste element de x-coördinaat is. Je gebruikt dus je kennis van hoe punten worden voorgesteld midden in je code. Het spreekt vanzelf dat dit het moeilijk maakt om de voorstelling van punten aan te passen, gezien je dan ook alle plaatsen in de code waar je van deze kennis bent uitgegaan moet opzoeken en wijzigen. Dit los je natuurlijk op door abstracties te maken die het gebruik en manipulatie van een concept in je code afschermt van de interne representatie van dat concept. Zodoende ben je in staat op elk moment voor een andere representatie te kiezen zonder dat dit enig gevolg heeft voor de code die gebruik maakt van dat concept. In ons voorbeeld kan je dan de voorstelling van het concept “3-dimensionaal punt” wijzigen naar een lijst met de coördinaten in omgekeerde volgorde, een vector enzovoorts.

Sommige studenten introduceren als “abstracties” in hun code de volgende “abstracties” voor `car` en `cdr`:

```racket
(define geef-eerste car)
(define geef-rest cdr)
```

, zodat men voortaan de volgende expressie kan uitvoeren:

```racket
(if (= (geef-eerste point) 1)
    (display "x-coördinaat is 1")
    (display "x-coördinaat is niet 1"))
```

Dit is een zeer povere “abstractie”, het is gewoon een herbenoeming van `car` en `cdr`. Het probleem hiermee is het volgende: stel je voor dat je naast punten in je code ook nog stacks hebt, die je ook voorstelt met lijsten. Omdat je weet dat deze zijn voorgesteld als lijsten, gebruik je overal in je code waar het je bovenste element van de stack nodig hebt dezelfde `geef-eerste` functie, omdat wij je gewaarschuwd hebben voor het gebruik van teveel `car`. Als je nu later de voorstelling van punten wil aanpassen zodat een vector en niet langer een lijst gebruikt wordt en je de implementatie van de `geef-eerste` functie wil aanpasse zodat deze het eerste element van de vector geeft, werken plots je stacks niet meer! Je moet immers geen abstracties maken voor `car` en `cdr`,

je moet functies voorzien voor het gebruik en de manipulatie van de concepten in je systeem.

Zo'n 'concept' (technisch : een datatype), tezamen met alle functies die nodig zijn om het concept te manipuleren (aanmaken, wijzigen, inhoud opvragen enzovoorts) noemen we een *ADT* (Abstract Data Type). De voorstelling van zo'n ADT wordt nooit naar buiten toe vrijgegeven. Daarom heet het datatype nou precies abstract!

Om terug te komen op het voorbeeld van hierboven, hierna volgt een implementatie van het ADT `punt`. Merk op dat je dus een andere implementatie voor hetzelfde punt ADT zou kunnen maken, met een andere interne voorstelling voor punten, zonder dat je de code die punten gebruikt moet aanpassen.

Racket
```
(define (make-point x y z)
    (mcons x (mcons y (mcons z '()))))

(define (give-x-coordinate point)
    (mcar point))

(define (give-y-coordinate point)
    (mcar (mcdr point)))

(define (give-z-coordinate point)
    (mcar (mcdr (mcdr point))))

(define (set-x-coordinate! point new-value)
    (set-mcar! point new-value)
    point)

(define (set-y-coordinate! point new-value)
    (set-mcar! (mcdr point) new-value)
    point)

(define (set-z-coordinate! point new-value)
    (set-mcar! (mcdr (mcdr point)) new-value)
    point)
```

De implementatie van ADTs is dus de enige plaats waar `car` en `cdr` mogen voorkomen, en enkel voor de manipulatie van de interne voorstelling van dat ADT. In de rest van je code gebruik je de functies die je voor het ADT gedefinieerd hebt. Als je in ons voorbeeld ook een ADT `stack` definieert en implementeert, en overal de correcte functies op punten en stacks gebruikt in plaats van `car`, `cdr`, `geef-eerste`, `geef-rest` enz. kan je de voorstelling van beide concepten aanpassen zonder dat dit een invloed heeft op de implementatie van het andere concept.

Merk op dat een soortgelijke eis natuurlijk ook geldt voor de operaties op andere primitieve datatypes in Scheme buiten lijsten. Ook het rechtstreeks gebruik van `vector-ref`, `vector-set!` etc. op de representatie van een "abstractie" is uit den boze wanneer je dit doet buiten de implementatie van de operaties van die "abstractie". Dit betekent immers dat je de abstractie doorbreekt en eigenlijk helemaal geen abstractie hebt gemaakt.

Abstract Data Type vs. Object

Eis: Maak het onderscheid tussen ADT's en objecten.

Studenten denken al te vaak dat, als ze een object maken in Scheme, ze meteen een ADT hebben, en omgekeerd, dat alle objecten ADT's zijn. Hiermee willen we dit misverstand uit de wereld helpen: niet alle ADT's zijn objecten, en zeker niet alle objecten zijn ADT's.

Een object, in Scheme meestal geïmplementeerd door een `lambda` waarnaartoe berichtjes gestuurd kunnen worden ter bevraging en wijziging van de interne toestand, is slechts één mogelijk strategie om een ADT te implementeren. Een ADT kan even goed geïmplementeerd worden met "gewone" functies, zoals hierboven werd geïllustreerd met het `punt` ADT.

Laten we even de vaak gebruikte term ADT uit elkaar trekken om een duidelijker beeld te geven van wat het nu juist is. Het is een *datatype*. Enkele voorbeelden van datatypes: `integers`, `strings`, `characters`, `booleans` enz. In de meeste programmeertalen zijn voorgaande *primitieve datatypes*, d.w.z. dat ze met de taal zijn meegeleverd, ze behoren tot de datatypes die standaard aangeboden worden door je programmeertaal.

Het is natuurlijk mogelijk om zelf nieuwe data types te implementeren. Zo zijn er de *abstracte datatypes*: het zijn datatypes die abstract zijn in die zin dat je met het datatype ook alle functies ter manipulatie ervan meegeleverd krijgt; je hoeft hierbij dus helemaal niet te weten hoe zo'n abstract datatype (intern) eigenlijk werkt, hoe het geïmplementeerd is; je kan het wel gebruiken.

Een stack is een goed voorbeeld van zo'n ADT. Om een stack te gebruiken hoef je niet te weten of een vector, een lijst, of misschien nog meer exotischer datastructuren gebruikt werden.

In het kort kunnen we dus zeggen dat objecten (en hun constructoren) een techniek zijn om ADT's te implementeren. Maar je kan ADT's evengoed op een andere manier implementeren (zie het `punt` ADT voorbeeld in de vorige sectie) en je kan de objecttechniek met dispatchers even goed gebruiken om andere abstracties te maken dan ADT's te implementeren (bijvoorbeeld het scheiden van een parser en een scanner).

Als je twijfelt of een bepaald concept een ADT is, stel jezelf dan de volgende vragen: moet ik weet hebben van de interne representatie om het 'ding' te kunnen gebruiken, kan het zin hebben om het 'ding' als parameter mee te geven met een functie (net zoals dat met gewone datatypes gebruikelijk is). Als je op de eerste vraag 'neen', en op de

tweede vraag ‘ja’ kan antwoorden, dan is dit een sterke aanwijzing dat je met een ADT te maken hebt.

Abstract Data Type vs. Datastructuur

Heel veel studenten halen de concepten Datastructuur (DS) en Abstract Data Type (ADT) door elkaar. Nochtans zijn dit twee afzonderlijke begrippen. De verwarring ervan heeft echter dikwijls vieze code als gevolg. Dikwijls wordt geprogrammeerd op basis van datastructuren daar waar er eigenlijk zou moeten geprogrammeerd worden op basis van ADT’s. De omgekeerde “fout” zien we spijtig genoeg veel minder.

Het verschil is dat een datastructuur programmeertaal-afhankelijk is daar waar een ADT dat niet is. Een ADT is zoals reeds gezegd een type dat abstract is; je weet dus niet wat de interne representatie is van dat type. Dit kan zowel gaan over een ingebouwd type in je programmeertaal (in Scheme bijvoorbeeld is het onmogelijk te weten te komen wat de interne representatie is van characters; dit type is dus even abstract als de ADT’s die je zelf maakt) als over een eigenhandig gebouwd ADT dat intern is opgebouwd uit een heel ingewikkelde structuur van data elementen (een datastructuur dus). De basisidee is echter dat de “gebruiker” van het ADT enkel de naam van het type kent en de operaties die op dat type gedefinieerd zijn. Zo kan het ADT `bit` in één of andere programmeertaal (met operaties `not`, `and`, `or`) zowel geïmplementeerd zijn als een “echte” bit in de machine, als een ingewikkelde datastructuur die bijvoorbeeld bijhoudt hoe dikwijls er vandaag al `not` is op toegepast. Je ziet het: de gebruiker ziet enkel de abstractie: een ding dat 2 waarden kan aannemen (een bit) samen met de abstracte operaties die er door de ontwerper van het ADT op gedefinieerd zijn. Of dat ding nu “elementair” dan wel m.b.v. een datastructuur gerepresenteerd werd is niet te zien aan het ADT.

Een datastructuur is zoals het woord het aangeeft, een structuur van data. Hiervoor ben je volledig afhankelijk van de manieren die je programmeertaal je aanbiedt om data “samen te lijmen” tot ingewikkeldere structuren. In Scheme zijn de manieren om datastructuren te bouwen dus cons-cellen, vectoren of records. In C bijvoorbeeld zijn dat `pointers`, `arrays` en `structs`.

In het kort: Om datastructuren te bouwen heb je de datastructureringstechnieken (constructors, etc…) van je taal nodig. Om ADT’s te bouwen heb je de abstractietechnieken van je taal nodig (hoe kan je een naam geven aan een ding!). Natuurlijk wordt heel dikwijls een ADT intern door een datastructuur gerepresenteerd, maar dat staat hier in feite los van.

Het probleem is dat veel studenten werken met behulp van datastructuren i.p.v. met ADT’s. Het voorbeeld hierboven illustreert het probleem: de student werkt in zijn hoofd (en in zijn code dus ook) met “lijsten” (en begint hierop zomaar `car` en `cdr` toe te passen) en niet met “punten”.

Booleans Zijn Volwaardige Datatypes

Sommige programmeurs met kennis van andere programmeertalen (bijvoorbeeld de taal “C”) zien booleans niet als volwaardige datatypes, dit omdat ze inderdaad in sommige andere talen niet als volwaardig worden beschouwd (in “C” bijvoorbeeld worden de getallen 0 en 1 gebruikt om booleaanse waarden voor te stellen).

Dit resulteert dan in code als:

```racket
(if (eq? a-boolean #t)
    (display "true")
    (display "false"))
```

daar waar

```racket
(if a-boolean
    (display “true”)
    (display “false”))
```

uiteraard netter en bovendien performanter is. De ergste instantie van dit verkeerd begrip die we al tegengekomen zijn ziet er als volg uit:

```racket
(define functie (lambda (args) … (if <test> #t #f))
```

Dit is natuurlijk klinklare onzin en moet als volgt worden gedaan:

```racket
(define functie (lambda (args) … <test>)
```

Codeduplicatie

Eis: Indien je in je code 2 of meer functies opmerkt die identiek gelijk zijn, op enkele details na, maak dan een algemenere functie. Maak eventueel gebruik van hogere orde abstracties.

Duplicatie kan zowel binnenin de body van een functie optreden als over verscheidene functies heen. Hieronder zien we een voorbeeld van een simpele functie waar een score aan een lijst dient toegevoegd te worden. Als de waarde die toegevoegd dient te worden groot genoeg is, worden er nog bonuspunten toegekend. Vaak zijn studenten geneigd om de body te starten met een `if`-expressie, of gelijkaardige controlestructuren om te bepalen wat er vervolgens dient te gebeuren. Echter zorgt het vroegtijdig introduceren van zulke structuren vaak voor herhaalde instructies met lichte verschillen:

```racket
; Slecht: duplicatie van structuur: 'cons' en 'rest-van-lijst'
(define (voeg-score-toe score rest-van-lijst)
  (if (> score 50)
      (cons (+ score 10) rest-van-lijst)
      (cons score rest-van-lijst)))
```

We zien in dit voorbeeld dat in beide “takken” van de `if`-expressie de lijst wordt uitgebreid m.b.v. `cons`, maar met een verschillende waarde. Het valt dus op dat de functie altijd een lijst dient uit te breiden, ongeacht de uitgevoerde test in de controlestructuur. Kortom: het opbouwen van de lijst staat in feite los van de controlestructuur. Idealiter zorg je er dus voor dat de controlestructuur slechts focust op de waarde bepalen die toegevoegd dient te worden aan de lijst. Dit kan je bereiken door eerst te beginnen met de essentie van de functie (de lijst uitbreiden), en de controlestructuur die de feitelijke waarde dient te bepalen, binnenin die expressie pas toe te passen:

Racket
```
; Beter: De if-expressie bepaalt alleen de waarde
(define (voeg-score-toe score rest-van-lijst)
  (cons (if (> score 50) (+ score 10) score)
        rest-van-lijst))
```

In projecten komen we regelmatig het zogenaamde “knip-en-plak-hergebruik” tegen. Knip-en-plak-hergebruik is een verbloemde term voor codeduplicatie, en moet ten allen tijde vermeden worden. Als je een aantal gelijkaardige functies nodig hebt, maar toch verschillen ze allemaal net ietsje, dan is een makkelijke en voor de hand liggende optie de functie een aantal keer te kopiëren, vervolgens de functies nieuwe namen te geven en de nodige kleine aanpassingen te maken. Deze manier van werken heeft gevolgen voor de herbruikbaarheid, en de aanpasbaarheid van de code: door te kopiëren introduceer je duplicatie en als later jij of anderen aanpassingen dienen te maken, dan dien je niet op 1 plaats te doen, maar op zoveel plaatsen als je je ene oorspronkelijke procedure gedupliceerd hebt.

In Scheme zijn functies zogeheten *first class citizens*, wat wil zeggen dat je de functies net zoals alle andere waarden kan gebruiken als argument van een functie of kan toekennen aan een variabele. Dit kan enorm handig zijn om codeduplicatie te vermijden. Stel dat je in Scheme een evaluator voor een programmeertaal ABC aan het schrijven bent, en je bent toegekomen aan de evaluatiefuncties voor `+`, `-`, `*` en `/`. Al deze functies moeten eigenlijk hetzelfde doen: ze krijgen getagde waarden binnen (namelijk ABC-getallen), moeten deze getagde waarden omzetten naar waarden waarmee je in Scheme kan werken (in ons geval zullen dat Scheme getallen zijn), past vervolgens de gevraagde bewerking toe, zet het resultaat terug om in ABC waarden, en geeft deze waarde naar buiten.

Racket
```
(define (eval-plus argument1 argument2)
    (number->abc (+ (abc->number argument1)
                    (abc->number argument2))))

(define (eval-min argument1 argument2)
    (number->abc (- (abc->number argument1)
                    (abc->number argument2))))
```

```
(define (eval-div argument1 argument2)
    (number->abc (/ (abc->number argument1)
                    (abc->number argument2))))

(define (eval-multiplier argument1 argument2)
    (number->abc (* (abc->number argument1)
                    (abc->number argument2))))
```

In plaats van door knip-en-plak-hergebruik nu de functies `eval-plus`, `eval-min`, `eval-div`, `eval-multiplier` te schrijven, kan je één functie `eval-operator` maken met parameters `argument1`, `argument2`, `operator` en die functie gebruiken om de andere vier functies te implementeren. De parameter `operator` kan in dit geval gewoon de gewone Scheme functie zijn (`+`, `-`, `*` en `/`), maar kan in meer gecompliceerde gevallen net zo goed een zelfgedefinieerde functie zijn. Deze techniek noemt men *hogere-orde-abstractie*.

```
(define (eval-operator argument1 argument2 operator)
    (number->abc (operator (abc->number argument1)
                           (abc->number argument2))))

(define (eval-plus argument1 argument2 )
    (eval-operator argument1 argument2 +))

(define (eval-min argument1 argument2 )
    (eval-operator argument1 argument2 -))

(define (eval-div argument1 argument2)
    (eval-operator argument1 argument2 /))

(define (eval-multiplier argument1 argument2)
    (eval-operator argument1 argument2 *))
```

Racket

Performantie en Geheugengebruik

Eis: Spring efficiënt om met berekeningstijd en geheugen, maar niet ten koste van duidelijke code.

Flagrante foutieve keuzes qua performantie worden niet meer geduld in tweede bachelor. Moet je bijvoorbeeld een lijst sorteren,gebruik dan geen *bubble sort*, want je weet dat er efficiëntere algoritmes beschikbaar zijn. Hoewel lijsten bijzonder handige datastructuren zijn, zijn ze niet altijd en overal de beste oplossing. Vergeet niet dat de meest gebruikte operaties op een vector een constante tijdscomplexiteit hebben die onafhankelijk is van de lengte van de vector, dit in tegenstelling tot lijsten. Verder zijn er

veel andere ADT's, allen geoptimaliseerd voor bepaald gebruik en bepaalde situaties: `stack`, `queue`, `priority-queue`, `B+-tree`, `dictionary`, `hash-tabel` enz.

Als je een ADT of datastructuur kiest om bepaalde elementen bij te houden, vraag je dan af hoe deze het meest gebruikt zullen worden. Zullen er bijvoorbeeld weinig nieuwe elementen worden bijgevoegd, maar wel veel elementen opgevraagd worden, kies dan voor een datastructuur die geoptimaliseerd is voor opvraging en niet voor toevoegingen. Ook simpele zaken, zoals het meermaals `car`'en van dezelfde lijst, omdat je het eerste element verschillende keren nodig hebt in een functie, zijn makkelijk te vermijden door met `let` het element aan een variabele te binden.

Ook geheugengebruik moet beperkt worden tot het nodige. Hash-tabellen van 50.000 hokjes waar maar 100 elementen worden aan toegevoegd zijn geheugenverkwisting. Let ook op het veelvuldig aanmaken van datastructuren dat tot overbodige *garbage collection* leidt. Een typisch voorbeeld hiervan is het schrijven van een scanner voor een programmeertaal. Er wordt een vector opgesteld met voor alle ASCII-karakters een uit te voeren functie, en er is een functie die gewoon een karakter neemt, deze omzet naar ASCII-waarde, en daarmee de gepaste functie in de vector opzoekt en uitvoert. Als je nu de vector binnen de body van deze functie aanmaakt, dan wil dat dus zeggen dat je iedere keer dat je de functie aanroept, 256 maal 4 bytes — oftewel 1 kilobyte geheugen — in gebruik genomen wordt, dat na het verlaten van de functie nooit meer gebruikt wordt en dus rijp is voor garbage collection. De scanner een groot bestand laten inlezen zal tot veelvuldige garbage collects leiden daar het geheugen na een aantal karakters in te lezen vol komt te zitten met deze nutteloos-geworden karakteropzoekingsvectoren. Wanneer de vector buiten de functie gedefinieerd wordt, moet deze slechts één maal effectief aangemaakt worden wat heel wat garbage collects in de toekomst uitspaart. Let wel dat dit niet betekent dat je daarom voor deze vector maar een globale variabele moet gebruiken, er zijn nettere manieren om dit op te lossen door de functie te definiëren binnen een eigen omgeving door middel van een `let`. De opzoekingsvector en het voorbeeld van de hash-tabellen zijn echter slechts twee voorbeelden van onnodige geheugenverkwisting; er zijn er tal van andere waarvoor je zelf dient te waken.

Wees je ervan bewust dat iedere keuze van algoritme, datastructuur, interne representatie voor ADT's enz. zal bekeken worden in functie van performantie en geheugengebruik, en dat je ten allen tijde je keuzes zult moeten kunnen verantwoorden.

Naamgeving en Andere Codeerconventies

Eis: Gebruik duidelijke naamgevingen in je code, die de bedoeling van de variabele of functie ook anderen duidelijk maakt.

Wees je er steeds van bewust dat anderen je code zullen lezen, en variabelen met namen als `i`, `tijdelijk`, `foo`, `temp`, `bla`, `aux`, `parameter`, `flcntr` enz. de leesbaarheid van je code allerminst bevorderen. Zelfs al gebruik je gauw even een variabele om te debuggen en ben je van plan om die variabele later terug weg te doen, dan nog

geef je die best een zinvolle naam. De beroemde 'opkuisfase' komt er toch bijna nooit, waardoor je code dan alsnog volstaat met betekenisloze variabelenamen.

Wees ook consistent in je naamgeving. Gebruik steeds dezelfde taal om je parameters/functies te benoemen: gebruik dus bijvoorbeeld altijd Nederlands of Engels, geen mengelmoes van de twee. Gebruik ook steeds dezelfde regels voor naamgeving: met of zonder koppeltekens, grote letters in een functienaam bij het begin van ieder nieuw woord enz. Je kan voor verschillende soorten functies, variabelen en dergelijke natuurlijk andere naamgevingen gebruiken, bijvoorbeeld interne variabelen van ADTs met kleine letters en globale variabelen in hoofdletters, zolang je dit maar consistent doet en je je aan je eigen regels houdt.

Houd je verder ook aan bestaande codeerconventies. Moet je aan een bestaand stuk code wijzigingen of uitbreidingen aanbrengen, onderzoek dan de methode van naamgeving van de oorspronkelijke programmeur, en probeer hier zo consistent mogelijk mee verder te gaan. Verder zijn er in de meeste programmeertalen bepaalde conventies. In Scheme bijvoorbeeld is het gebruikelijk om de namen van predicaten te laten eindigen op een vraagteken (bijvoorbeeld `is-empty?`), en die van destructieve operaties te laten eindigen op een uitroepteken (bijvoorbeeld `set-x-coordinate!`).

Geen Vastgebakken Waarden

Eis: Zorg dat er nergens in je code vastgebakken waarden voorkomen.

Voorbeelden van vastgebakken strings zouden kunnen zijn: DNS-namen van machines waarnaartoe je wil connecteren, URL's enz. Vastgebakken getallen komen meestal voor als boven- of ondergrenzen (bijvoorbeeld het maximum aantal spelers bij een bordspel), aantallen voor iets (bijvoorbeeld het aantal agents dat in een systeem actief is) enz.

Je kan dit simpelweg vermijden door de hard gecodeerde waarden als constanten te definiëren. Zorg er wel voor dat je code nergens steunt op een specifieke waarde van een constante (je mag natuurlijk wel constanten gebruiken). Als je bijvoorbeeld een bordspel implementeert, en je wil (initieel) maximum 6 spelers ondersteunen, zorg dan dat je code er nergens vanuitgaat dat er maar 6 spelers zullen zijn (bijv. door ergens expliciet 6 vectoren, 1 per speler, aan te maken om bepaalde zaken bij te houden). Dit verhindert aanpasbaarheid.

Wanneer je vastgebakken waarden feitelijk configuratiegegevens betreffen, dan is het gebruik van een configuratiebestand ook een handige oplossing. Het heeft, net zoals het gebruik van (globale) constanten trouwens, het voordeel dat je op slecht 1 plaats veranderingen moet aanbrengen als dat nodig zou zijn. Op de verdediging kan er je weinig ergers overkomen dan een aantal foutmeldingen dat bijvoorbeeld een bestand niet kan worden gevonden, waarna je wanhopig in je code moet gaan zoeken om te vinden waar je overal de naam en/of locatie van dat bestand vastgebakken hebt.

Geen Globale Variabelen

Eis: Gebruik enkel globale variabelen als je niet anders kan.

Het gebruik van globale variabelen is weliswaar in sommige gevallen nodig, en zelfs nuttig (zie bijvoorbeeld het puntje over vastgebakken waarden, of het puntje over performantie), maar in het algemeen maken teveel globale variabelen je code onduidelijk en onleesbaar. Het gebruik van globale variabelen midden in je code zorgt er bovendien voor dat je code niet te hergebruiken valt. Als je toch een globale variabele gebruikt in een functie, dan is het een goed idee deze mee te geven als parameter. Zo steunt de betreffende functie niet op het bestaan van deze globale, en kan aldus hergebruikt worden.

Het is bovendien een goed idee om voor globale variabelen een aparte naamgevingsconventie te gebruiken (bijvoorbeeld volledig in hoofdletters), zodat de lezer van je code meteen ziet dat iets globaal gedefinieerd is, zonder dat hij minutenlang moet gaan zoeken waar die waarde exact vandaan komt.

Gebruik de Mogelijkheden van De Programmeertaal

Informeer je over de programmeertaal waarin je aan het programmeren bent. Dit doe je aan de hand van het raadplegen van de beschikbare documentatie van een programmeertaal. Voor Racket kan je hier een kijkje nemen.

Sommige programmeertalen beschikken over krachtige mechanismen, die in andere programmeertalen niet bestaan. Zo bevat Racket bijvoorbeeld een manier om klassen en object te definiëren, en beschikt Scheme beschik over symbolen, die intern gerepresenteerd worden als unieke geheugenadressen. Hierdoor is het vergelijken van symbolen veel performanter dan het vergelijken van strings, omdat bij strings elk karakter afzonderlijk moet vergeleken worden, terwijl om symbolen te vergelijken gewoon de geheugenadressen vergeleken moeten worden. Als je dus een identiteit nodig hebt in Scheme, dan is er niets beter dan een symbol. Omdat je in andere talen dit mechanisme niet hebt, worden daar andere manieren gehanteerd om identiteiten voor te stellen.

Korte Procedures

Eis: Hou je procedures kort. Procedures die in hun geheel op het scherm passen zijn makkelijker te begrijpen. Op papier zouden procedures van een A4 lang uitzonderlijk moeten zijn, langer is zeker uit den boze.

Te lange procedures maken je code onleesbaar, en wijzen bijna steeds op het feit dat je niet genoeg geabstraheerd hebt. De informele richtlijn die je kan hanteren is dat een procedure die niet op één A4 blad past echt wel te lang is.

Let er wel voor op dat je geen “abstracties” maakt waarbij je een veel te grote procedure opsplitst in kleinere, die dan steeds op het einde “de volgende” aanroepen. De bedoeling van korte procedures is dat de lezer van je code elke procedure in één oogopslag

kan begrijpen. Dit is duidelijk niet het geval als je een lange procedure sequentieel opsplitst in n korte procedures die mekaar achtereenvolgens aanroepen. Indien je een procedure hebt die te lang is, moet je zoeken naar abstracties in deze procedure, de abstracties eruit lichten en vervolgens de weggenomen dingen op zich als procedure oproepen.

Code met “mooie” abstracties bestaat dan ook uit een verzameling procedures die mekaar *hiërarchisch* oproepen en niet uit een verzameling procedures die mekaar sequentieel oproepen.

Codehergebruik van Derden

Eis: Bij hergebruikte code dient steeds verwezen te worden naar de bron. Bronvermeldingen documenteer je best in je design document, alsook in commentaren in de broncode.

Het is toegelaten, en wordt zelfs aangemoedigd, om code van anderen te hergebruiken, tenzij het in de opgave expliciet anders vermeld staat. We hebben het hier uiteraard niet over het kopiëren van collega-studenten die dezelfde projectopgave hebben als jij (voor alle duidelijkheid, dit is niet toegelaten), doch wel over het gebruiken van bijvoorbeeld een standaardimplementatie van een sorteeralgoritme of iets dergelijks. Het verschil tussen hergebruiken van code en stelen van code, is natuurlijk de bronvermelding. Bij ieder stuk hergebruikte code dien je te vermelden waar de code vandaan komt, zodat het duidelijk is dat je deze code niet zelf geschreven hebt. Doe je dat niet, dan pleeg je plagiaat, met alle gevolgen van dien als we dit ontdekken. Dit is eveneens van toepassing op code gegenereerd door AI-tools.

Let er wel op dat hergebruikte code nooit een excuus kan zijn voor bugs of het slecht werken van jouw programma. Je bent evenzeer verantwoordelijk voor de hergebruikte code als voor je eigen code! Crasht je programma dus door een bug in de hergebruikte code, dan moet je ofwel die bug er uithalen, ofwel de code niet gebruiken en gewoon zelf het nodige implementeren. Hetzelfde geldt voor slechte implementaties: gebruik die gewoon niet, doe je het wel dan kan je evenzeer punten verliezen doordat je bijvoorbeeld een niet-efficiënte sorteeralgoritme (her)gebruikt hebt.

Om kort samen te vatten, de hergebruikte code moet aan dezelfde kwaliteitseisen voldoen als je eigen code, anders gebruik je die code niet.

Scheiding van Belangen

“Scheiding van Belangen” (*“Separation of Concerns”*) is een ingeburgerd concept in de software-architectuurwereld. Dit begrip hier volledig uitleggen is onmogelijk (er zijn vakken later in je curriculum die hierover handelen), maar we zullen toch proberen je het basisidee uit te leggen, zodat je dit kunt toepassen in je project.

Het wordt algemeen aanzien als een goede programmeerstijl om te proberen de verschillende belangen van je softwaresysteem te identificeren, en vervolgens zo onafhankelijk mogelijk van elkaar te implementeren. Voorbeelden van zo’n belangen

kunnen zijn: communicatie (onderling, via netwerk), output (naar de gebruiker toe), persistentie enzovoorts.

Door de verschillende belangen zo goed mogelijk gescheiden te houden, zorg je ervoor dat je code in een later stadium makkelijk aanpasbaar is. Wil je bijv. de onderlinge communicatie tussen de verschillende software-componenten veranderen van oproepen via *sockets* naar oproepen via een TCP/IP netwerk, dan zou dit geen probleem mogen vormen indien je dit vooraf als een bekommernis had geïdentificeerd, en het aldus zo onafhankelijk mogelijk van de rest van je code geïmplementeerd had.

Waar bovenstaande uitleg voor je programmeerproject al zeker kan toegepast worden is uitvoer en debugging. Vaak vinden we oproepen van `display` terug overal verweven in de code. Stel dat je nu later van uitvoerformaat wil veranderen (bijvoorbeeld grafisch werken en niet langer tekstueel), dan mag je manueel al je code gaan overlopen om overal je oproepen van `display` te gaan vervangen door oproepen van functies in de bibliotheek van grafische functies.

Je introduceert voor de bekommernissen 'uitvoer en debugging' beter het ADT "tracer", dat je toelaat de uitvoer naar believen in en uit te schakelen. We geven hier alvast onafgewerkte voorbeeldcode voor zo'n tracer ADT, maar het is aan jou om dit uit te breiden en beter aan te passen aan je noden:

Racket

```
(define (make-tracer)
  (define activated #t)

  (define output-port (current-output-port))

  (define (logger . args)
    (for-each (lambda (x)
                (display x output-port)
                (display " " output-port))
              args)
    (display #\newline output-port)
    (flush-output output-port))

  (define (set-tracer-on!)
    (set! activated #t)
    (log-info "Tracer activated"))

  (define (set-tracer-off!)
    (log-info "Tracer de-activated")
    (set! activated #f))

  (define (log-info . args)
```

```
    (when activated
      (apply logger (cons "INFO   " args))))

  (define (dispatch-tracer msg . args)
    (case msg
      [(set-tracer-off!) (set-tracer-off!)]
      [(set-tracer-on!) (set-tracer-on!)]
      [(log-info) (apply log-info args)]
      [else (error 'dispatch-tracer
                   "Message not Understood: ~a"
                   msg)]))
  dispatch-tracer)
```

Debugging & Testing

Eis: Gebruik tracers om debugging output te verzorgen.

Zoals hierboven reeds vermeld is een tracer ADT enorm handig om je debugging code te scheiden van de werkelijke code van je programma. Een handige operatie om eveneens in je tracer op te nemen is de volgende:

Racket
```
(define (show-parameter message the-parameter)
  (displayln message)
  (displayln the-parameter)
  the-parameter))
```

Hiermee kan je “verdachte” parameters (dit zijn parameters die om onbegrijpelijke redenen “verkeerde” waarden aannemen) eventjes tonen, zonder dat je code moet gaan aanpassen of dat dit de werking van je programma verstoort.

Verder nog een woordje over debuggen in het algemeen. Zoals jullie waarschijnlijk wel weten bestaan er twee methoden om procedures/functies te debuggen: *white box testing* en *black box testing*.

White box testing bestaat eruit de code aandachtig te bestuderen, te begrijpen wat de code doet en op die manier logische of syntactische fouten te proberen ontdekken.

Black box testing bestaat eruit de procedure/functie als een zwarte doos te beschouwen, verschillende waarden voor de parameters in te vullen, en te zien of het resultaat gelijk is van wat men als resultaat van de procedure verwacht. Hierbij is het natuurlijk aan te raden om, naast standaardinput, ook randvoorwaarden te bedenken (een typisch voorbeeld zou zijn een lege lijst ingeven in procedures die lijsten verwachten). Hoe meer representatieve steekproeven genomen worden, hoe beter het idee van correctheid van de procedure. Let wel, black box testing biedt nooit een garantie voor correctheid van de procedure.

Eis: Voor elk ADT moet een aparte test file voorzien worden welke werking van de kernfunctionaliteit van het ADT verifieert met behulp van unit tests.

Een goede manier om aan (black box) testen te doen is gebruik te maken van de *unit testing* methode. In deze methode wordt elk individueel stuk code (bijv.: elke procedure) van een specifiek stuk testcode voorzien. In de loop van de eerste semester zullen wij een demonstratie geven van unit testing in (Dr)Racket met behulp van RackUnit.

Documentatie

Eis: Documenteer je code zodat ook anderen deze kunnen begrijpen.

Niet-gedocumenteerde code is per definitie slechte code. Hoe goed je je abstracties ook maakt, hoe duidelijk je je naamgeving ook kiest, hoe proper je codeerconventies ook zijn, je code zal steeds behoefte hebben aan duidelijke en consistente documentatie als je wil dat derden je code op een vrij comfortabele manier kunnen volgen.[1]

Documenteer dus iedere niet-triviale procedure/functie en ADT. Beschrijf wat de procedure doet, en eventueel hoe dit gebeurt als dit niet meteen duidelijk zou zijn. Geef duidelijk aan wat de argumenten zijn, en wat de output van de procedure is.

Een voorbeeld:

Racket

```
; Beschrijving: ADT stack
; Argumenten: geen
; Output: stack object
; Messages:
; - push:
;     argumenten: element (het element dat we willen toevoegen)
;     output: het toegevoegde element
; - pop: enz.
; Commentaar: In deze implementatie van het Stack ADT wordt een
; vector-representatie gebruikt.
(define (make-stack) ...)
```

Stijlfouten voor Beginners

Eis: Vermijd stijlfouten waarvoor al in eerste bachelor gewaarschuwd werd.

In het verleden zijn er in de cursus “Structuur van Computerprogramma’s” een aantal stijlfouten gebundeld in een document die helaas door veel studenten gewoon genegeerd werden en nog steeds genegeerd worden. Dat document is beschikbaar op de Canvas-ruimte van dit vak. Herlees deze nog een aantal keer! Dit zijn fouten die op het

[1] Dit geldt natuurlijk niet voor code die geschreven werd om les mee te geven. Daar bestaat de documentatie uit de uitleg van de lesgever + aanwezigheid van de student in de les ☺.

einde van de tweede bachelor niet meer geduld worden. Vooral het veelvuldig vervuilen van code met `display`'s en het benoemen van iedere hulpprocedure als `iter` (daar waar helemaal geen iteratie te bespeuren valt) zijn alomtegenwoordig in studentencode.

Samenvatting en Slotopmerkingen

We hebben onderscheid gemaakt tussen drie globale eisen waar zowat elk softwaresysteem aan dient te voldoen: zijn functie vervullen, efficiënt omspringen met de middelen van computerapparatuur en makkelijk aanpasbaar en uitbreidbaar zijn. We hebben uitgelegd hoe je aanpasbaarheid en uitbreidbaarheid kan bereiken door de nodige abstracties te maken, hoe je hiervoor de notie van Abstract Data Type correct dient te gebruiken en hoe deze notie verschilt van de noties van objecten en datastructuren. Correct gebruik van ADT's impliceert dat je het gebruik van kennis over de interne representatie van het ADT beperkt tot de implementatie ervan, zodat je de interne representatie en de interne implementatie van de operaties kan aanpassen zonder de plaatsen in de code waar het ADT gebruikt wordt te moeten wijzigen. De impact van andere aanpassingen zoals het veranderen van boven- en ondergrenzen, configuratiegegevens enz. kan eveneens beperkt worden door kennis hierover af te schermen van alle plaatsen waar de grenzen en gegevens nodig zijn. Het vermijden van vastgebakken waarden is meteen ook één van de weinige goede redenen om globale variabelen te gebruiken. Ook codeduplicatie dient vermeden te worden omdat dit er eveneens toe leidt dat een aanpassing wijzigingen impliceert op veelvuldige plaatsten in de code.

Omdat een softwaresysteem niet enkel aanpasbaar moet zijn door de oorspronkelijke programmeur(s), maar ook door anderen, vereist aanpasbaarheid leesbaarheid van de code. Dit wordt al bereikt door de nodige abstracties te maken en scheiding van belangen toe te passen, maar vraagt tevens aandacht voor duidelijke naamgeving in de code. Grote procedures hiërarchisch verdelen opent ook de mogelijkheid om de onderdelen te benoemen, wat leesbaarheid verder ten goede komt. Omdat naamgeving alleen niet alles zegt is goede documentatie van ADT's echter nog steeds nodig. Ten slotte gaven we ook enkele eisen wat betreft codehergebruik van derden en beginnersstijlfouten die je dient te vermijden.

Voldoen aan de concretere eisen die we je gaven zoals het vermijden van het onnodig gebruik van `car` en `cdr` enzovoorts helpen je om te zorgen dat je code voldoet aan de drie globale eisen. Hou bij het schrijven van je code wel in gedachten dat die drie globale eisen zijn wat we voor ogen hebben en waar je project op geëvalueerd wordt.

Van de drie globale eisen is ervoor zorgen dat je systeem doet wat het moet doen de allerbelangrijkste, je code moet zeker voldoen aan minimumimplementatievereisten. Dit is een nodige, maar niet voldoende voorwaarde om te slagen! Aan aanpasbaarheid en uitbreidbaarheid hechten we namelijk ook zeer veel belang. In het geval van implementatie-extra's (dus niet minimum-implementatievereisten) beschouwen we het zelfs als belangrijker dan de eerste eis: een student die ons kan duidelijk maken dat zijn

code voldoende uitbreidbaar is om de extra's makkelijk te implementeren verdient van ons een hogere score dan een student die de extra's snel door wat onleesbaar gehack heeft bijgevoegd! Ten slotte is ook efficiëntie van zeer groot belang, maar wordt het als ondergeschikt beschouwd aan aanpasbaarheid en uitbreidbaarheid. De twee spreken elkaar soms tegen: één lange functie betekent minder functie-oproepen dan een korte functie die weer kleinere functies oproept, en betekent dus minder berekeningstijd. In dat geval is leesbaarheid en dus aanpasbaarheid belangrijker. En de twee spreken elkaar ook niet altijd tegen: goede abstracties maken betekent vaak dat je een implementatie kan maken die bijvoorbeeld vaak gebruikte operaties van de abstractie een lineaire in plaats van kwadratische tijdscomplexiteit geeft, wat een veel belangrijker efficiëntievoordeel is dan wat functie-oproepen uitsparen.

Ten slotte, we weten natuurlijk ook wel dat programmeren en het zoeken naar abstracties en mooie code geen exacte wetenschap zijn. Bovenstaande eisen zijn dus stuk voor stuk wel ooit, en ergens legitiem, te breken in een bepaald specifiek geval. Maar deze gevallen zijn echter zeldzaam. Zondigen tegen één van bovenstaande regels zal op je verdediging dan ook zwaar verdedigd moeten worden. Tenzij je een geldige reden hiervoor hebt, zal dit onvermijdelijk en serieus puntenverlies tot gevolg hebben.

We raden je tevens nog aan dit document tijdens de loop van je project af en toe eens te herlezen. Eenmalig lezen zal je waarschijnlijk weinig helpen. Neem het af en toe eens bij de hand en kijk of je code wel degelijk of nog steeds voldoet aan de eisen. Plak de belangrijkste punten desnoods met een Post-It op je computerscherm! Ook kan er inspiratie gehaald worden uit deze handleiding over het schrijven van leesbare Racket-code.